Lenguajes de Programación

Convocatoria Adelantada

Fecha: 15 de octubre de 2015

Advertencia Importante: Observa bien la sintaxis en los ejemplos. No se considerarán soluciones que no cumplan con las condiciones impuestas por el enunciado del ejercicio.

EJERCICIO 1 (2,5 puntos)

Sin definir otras funciones auxiliares, definir una función en lisp llamada **elemento_central** que dada una lista de enteros devuelva como resultado el elemento central de la lista, o NIL si el número de elementos fuera par.

```
> (elemento_central '(1 2 3 4 5 6 7 8 9 10))
NIL
> (elemento_central NIL)
NIL
> (elemento_central '(1 2 3 4 5 6 7 8 9))
5
```

EJERCICIO 2 (2,5 puntos)

Sin definir otras funciones auxiliares, define una función en LISP llamada **eliminar_central** que elimine el elemento central de una lista dada como parámetro. Si la lista está vacía o contiene un número par de elementos, el resultado deberá ser la lista original.

```
> (eliminar_central '(1 2 3 4 5 6 7 8 9 10))
(1 2 3 4 5 6 7 8 9 10)
> (eliminar_central NIL)
NIL
> (eliminar_central '(1 2 3 4 5 6 7 8 9))
(1 2 3 4 6 7 8 9)
```

EJERCICIO 3 (2,5 puntos)

Definir una función en lisp llamada **lista_replicados** que acepte una lista sin anidamientos y un número entero positivo *n*, y devuelva una lista cuyos elementos sean listas que contienen *n* veces el elemento que estaba en su misma posición en la lista original. Puedes utilizar las funciones adicionales que consideres necesarias.

```
> (lista_replicados '() 6)
NIL

> (lista_replicados '(1) 6)
((1 1 1 1 1 1))

> (lista_replicados '(1 3 2) 3)
((1 1 1) (3 3 3) (2 2 2))

> (lista_replicados '(1 3 2) 0)
(NIL NIL NIL)

> (lista_replicados '(1 3 2) 6)
((1 1 1 1 1 1) (3 3 3 3 3 3) (2 2 2 2 2 2))

> (lista_replicados '(1 2 3) 2)
((1 1) (2 2) (3 3))
```

EJERCICIO 4 (2,5 puntos)

Sin definir otras funciones auxiliares, define una función en LISP llamada **unary_functions** que acepte una lista sin anidamientos de valores numéricos y un número variable de funciones unarias. La función deberá devolver una lista con un elemento por cada una de las funciones unarias suministradas como parámetro. Cada elemento deberá contener el resultado de aplicar una de las funciones sobre cada uno de los elementos de la lista original, de manera que el primer elemento de la lista resultante contenga el resultado de aplicar la primera función sobre cada elemento de la lista, el segundo el resultado de aplicar la segunda y así sucesivamente.

```
> (unary_functions '(1 2 3) #'1+ #'1-)
((2 3 4) (0 1 2))

> (unary_functions '(1 2 3))
NIL

> (unary_functions '(1 2 3) #'1+)
((2 3 4))

> (unary_functions '() #'1+ #'1-)
(NIL NIL)
```